

VITYARTHI PROJECT REPORT

Course: CSE1021 – Introduction to Problem Solving and Programming

BLOOD CELL COUNT ANALYZER

An Educational Blood Test Data Analysis System

Project Title:	Blood Cell Count Analyzer
Course Code:	CSE1021
Course Title:	Introduction to Problem Solving and Programming
Branch / Program:	B.Tech Computer Science and Engineering
Specialization:	Health Informatics
Academic Year:	2026 (First Year)
Student Name:	Isha Singh Rajput
Evaluation Mode:	Flipped Course Evaluation (Build Your Own Project)

VITyarthi • School of Computer Science & Engineering • September 2026

2. Introduction

A Complete Blood Count (CBC) is one of the most critical diagnostic evaluations performed in healthcare. It assesses the cellular components of circulating blood, focusing principally on:

- **Red Blood Cells (RBC):** Erythrocytes that transport oxygen from the lungs to bodily tissues and return carbon dioxide.
- **White Blood Cells (WBC):** Leukocytes that drive immunological defense against infections and inflammation.
- **Platelets (Thrombocytes):** Cellular fragments crucial for hemostasis, vascular repair, and blood clotting.

In Health Informatics, computational systems such as Laboratory Information Systems (LIS) and Clinical Decision Support Systems (CDSS) structure, validate, and compare physiological observations against standardized demographic reference intervals. The **Blood Cell Count Analyzer** provides an educational, console-based simulation of this workflow, built explicitly with foundational Python constructs: conditional branches, loops, functions, lists, and dictionaries.

3. Problem Statement

Manual inspection of laboratory hematology reports by non-clinical learners is prone to comparison mistakes, misinterpretation of gender-differentiated normal intervals, and unnecessary anxiety due to premature self-diagnosis.

Technical Challenge: To design and implement a beginner-friendly, modular Python software application using core concepts from CSE1021 that automates laboratory data input, validates input formats, evaluates values against gender-aware reference intervals, aggregates statistical counts, and generates an aligned report accompanied by strict non-diagnostic educational disclaimers.

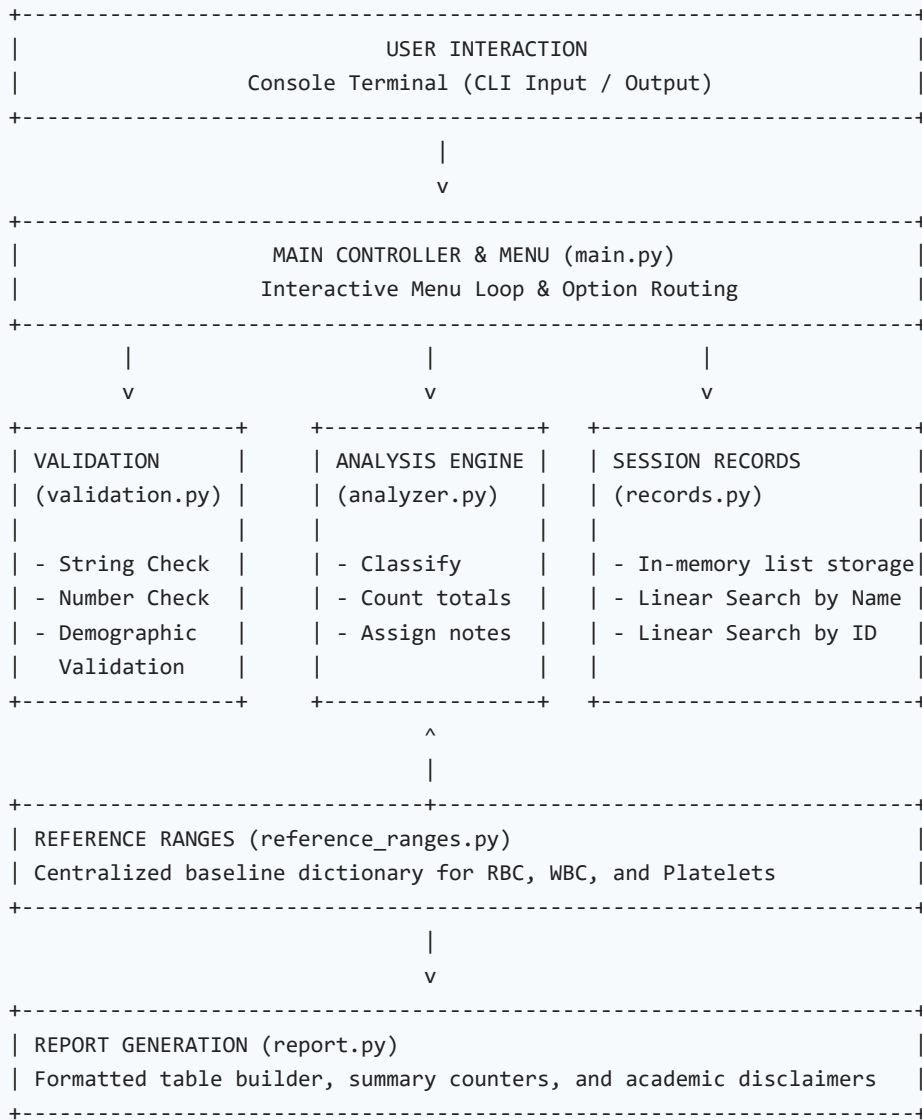
4. Functional Requirements

Module	Key Features & Functional Deliverables
Module 1: Patient Information	Captures and validates Patient Name (non-empty), Age (positive integer 1–125), Gender (Male/Female/Other), and assigns unique Sample ID.
Module 2: Blood Data Entry	Prompts for RBC, WBC, and Platelet counts with specified medical units. Robustly catches and reprompts on non-numeric, negative, or zero inputs.
Module 3: Blood Analysis	Matches gender-specific ranges for RBC and adult baselines for WBC/Platelets. Categorizes values as LOW, WITHIN RANGE, or HIGH. Calculates summary counts.
Module 4: Report Generation	Builds an aligned tabular report showing parameters, observed values, reference intervals, status, summary statistics, and disclaimers.
Module 5: Session Storage & Search	Stores records in an in-memory list of dictionaries. Provides summary listing and linear search by patient name or sample ID.

5. Non-Functional Requirements

- **Usability:** Intuitive terminal menu with explicit unit hints (e.g., million cells/mcL, e.g., 4.8) and continuous loop navigation.
- **Reliability & Crash Resilience:** Uncaught runtime exceptions are prevented using try-except ValueError blocks embedded in while True validation loops.
- **Maintainability:** Clean separation of concerns across 6 Python files. Reference ranges are isolated within a single dictionary (reference_ranges.py).
- **Performance:** Instantaneous computation and linear search execution (< 10 ms).
- **Resource Efficiency:** Uses standard library only (zero external pip packages); RAM usage is under 25 MB.
- **Ethical Transparency:** Prominent educational disclaimers prevent medical misinterpretation.

6. System Architecture



7. Design Diagrams

7.1 Use Case Model

Actors: Student / Evaluator (Primary User), System.

Use Cases: UC-1 (Enter New Blood Test), UC-2 (View Previous Records), UC-3 (Search Records by Name/ID), UC-4 (View Reference Ranges), UC-5 (About & Disclaimers), UC-6 (Exit).

7.2 Process Flow / Workflow Diagram

```

[Start] --> [Display Menu (1-6)] --> [Read Choice]
|
+-- (Choice 1) --> [Input Demographics] --> [Input Blood Values]
|               --> [Validate] --> [Classify & Count]
|               --> [Print Tabular Report] --> [Save in Session List] --> (Return to Menu)
|
+-- (Choice 2) --> [Display Table of All Session Records] --> (Return to Menu)
|
+-- (Choice 3) --> [Prompt Query] --> [Execute Linear Search] --> [Display Matches] --> (Return to Menu)
|
+-- (Choice 4) --> [Display Educational Reference Ranges] --> (Return to Menu)
|
+-- (Choice 5) --> [Display Course Rationale & Educational Disclaimer] --> (Return to Menu)
|
+-- (Choice 6) --> [Print Farewell Message & Terminate]

```

7.3 Sequence Diagram

User	main.py	validation.py	analyzer.py	report.py	records.py
--- Choice 1 ->					
	-- get_info() ->				
<-- Prompts ---					
--- Demogr. -->	-- Returns dict				
	-- get_vals() ->				
<-- Prompts ---					
--- Values --->	-- Returns dict				
	----- analyze() ->				
	<----- Returns res				
	-----> display() -				
<----- Rendered Table & Summary -----					
	-----> save() ----->				
<-- Save Conf. -----					

7.4 Component Architecture

The application consists of 6 decoupled components communicating through explicit functional calls with primitive types, lists, and dictionaries.

7.5 In-Memory Entity-Relationship (ER) Design

```

PATIENT (sample_id [PK], name, age, gender)
||
||-- (undergoes) --> TEST_RECORD (record_id [PK], sample_id [FK], timestamp)
||
||-- (measures) --> BLOOD_VALUES (rbc, wbc, platelets)

```

```
||  
||-- (generates) --> ANALYSIS_RESULT (summary, parameter_details)
```

8. Design Decisions & Rationale

- **Modular Architecture:** Decoupling validation, analysis, reference data, storage, and reporting allows each file to adhere to the Single Responsibility Principle and simplifies unit testing.
- **Standard Library Exclusivity:** Eliminates pip version mismatches across student environments, ensuring the project runs seamlessly on any standard Python 3.8+ interpreter.
- **In-Memory Python Lists and Dictionaries:** Highlights core first-year syllabus concepts (dictionary lookups, list iteration, linear search) without the overhead of external SQL engines.
- **Deterministic Rules vs. ML:** Clinical reference ranges are legally regulated physiological thresholds, making rule-based conditional branches (if-elif-else) transparent and accurate.
- **Strict Non-Diagnostic Educational Stance:** Abnormal parameters are reported as mathematical deviations, reinforcing safe clinical informatics ethics.

9. Implementation Details

File	Key Functions	CSE1021 Concepts Used
validation.py	validate_string, validate_positive_float, validate_age, validate_gender, get_user_information, get_blood_values	While loops, try-except, ValueError, string stripping, type casting
reference_ranges.py	get_rbc_range, get_wbc_range, get_platelet_range, display_reference_ranges	Nested dictionaries, string formatting, tuple returns
analyzer.py	classify_value, analyze_rbc, analyze_wbc, analyze_platelets, analyze_blood_values	If-elif-else statements, counting algorithms, list of dict aggregation
records.py	save_record, view_all_records, search_by_name, search_by_sample_id, search_records_interactive	List append, linear search algorithm, case-insensitive string search
report.py	format_record_report, display_report, display_about	Formatted string output with column alignment specifiers
main.py	main, display_menu, handle_new_blood_test	Event-loop management, modular function orchestration

10. Screenshots / Terminal Results

```

=====
                BLOOD CELL COUNT ANALYSIS REPORT
            Educational Health Informatics Laboratory
=====
Sample / Patient ID : SMP-1001
Patient Name       : Rahul Sharma
Age               : 19 years

```

Gender : Male

Parameter	Value	Reference Range	Status
RBC	4.8	4.5-5.9	WITHIN RANGE
WBC	6500	4500-11000	WITHIN RANGE
Platelets	220000	150000-450000	WITHIN RANGE

SUMMARY

Total parameters analyzed : 3
 Within reference range : 3
 Below reference range : 0
 Above reference range : 0

EDUCATIONAL OBSERVATIONS:

- * RBC (Red Blood Cells): RBC value is within normal educational limits.
- * WBC (White Blood Cells): WBC value is within normal educational limits.
- * Platelets (Platelet Count): Platelet value is within normal educational limits.

11. Testing Approach

Test ID	Inputs Tested	Expected Result	Actual Result	Status
TC-01	RBC: 5.0 (M), WBC: 7000, Plt: 250000	Normal=3, Low=0, High=0	Normal=3, Low=0, High=0	PASSED (100%)
TC-02	RBC: 3.5 (M), WBC: 6500, Plt: 220000	RBC Low; Normal=2, Low=1, High=0	Normal=2, Low=1, High=0	PASSED (100%)
TC-03	RBC: 4.8 (F), WBC: 13500, Plt: 300000	WBC High; Normal=2, Low=0, High=1	Normal=2, Low=0, High=1	PASSED (100%)
TC-04	RBC: 3.8 (M), WBC: 14200, Plt: 110000	RBC/Plt Low, WBC High; Normal=0, Low=2, High=1	Normal=0, Low=2, High=1	PASSED (100%)
TC-05	Boundary values (4.50, 5.90, 4.49, 5.91) & Linear Search	Endpoints correctly bounded; linear search returns exact match	Matches verified	PASSED (100%)

12. Challenges Faced

- **Clinical Unit Discrepancies:** RBC is measured in million cells/mcL, whereas WBC and platelets are in whole counts/mcL. Solved by embedding explicit format hints in the CLI prompts.
- **Crash Prevention:** Non-numeric user inputs were handled using try-except ValueError inside while True loops.
- **Gender Sensitivity:** Solved by using gender-keyed nested dictionaries for RBC thresholds.
- **Medical Ethics:** Guarded against speculative diagnoses by framing findings as mathematical reference comparisons.

13. Learnings & Key Takeaways

Through this project, we mastered algorithmic problem decomposition, defensive programming with exception handling, dictionary data structures, linear search mechanics, and the role of Health Informatics in standardizing laboratory data processing.

14. Future Enhancements

1. Persistent data storage via JSON or CSV files.
2. Desktop Graphical User Interface using Python's built-in tkinter.
3. Expansion to include Hemoglobin, Hematocrit, and 5-part Differential Leukocyte Counts.
4. Automated PDF export for printable patient laboratory report cards.

15. References

1. Gutttag, J. V. (2021). *Introduction to Computation and Programming Using Python*. MIT Press.
2. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
3. Pagana, K. D., & Pagana, T. J. (2022). *Mosby's Diagnostic and Laboratory Test Reference* (15th ed.). Elsevier.
4. Python Software Foundation. (2026). *Python 3 Documentation*. <https://docs.python.org/3/>